Capítulo 9. Estructura y layout

Los capítulos anteriores muestran las decenas de etiquetas XHTML disponibles para marcar y estructurar cada elemento individual de las páginas web: tablas, listas, enlaces, párrafos, imágenes, etc. Aunque combinando esas etiquetas es posible crear cualquier página web, no es posible hacer que las páginas muestren estructuras complejas.

La mayoría de páginas HTML disponen de estructuras complejas formadas por varias columnas de contenidos y otro tipo de divisiones. Utilizando exclusivamente HTML no es posible crear estas estructuras complejas, ya que es imprescindible emplear las hojas de estilos CSS.

No obstante, los estilos de CSS necesitan la ayuda de HTML/XHTML para crear los diseños más avanzados. En concreto, el código HTML se encarga de agrupar los elementos de la página en diferentes divisiones en función de su finalidad: la zona de la cabecera de la página, la zona de contenidos, una zona lateral para el menú y otras secciones menores, la zona del pie de página, etc.

La siguiente imagen muestra algunas de las zonas definidas en la página principal del sitio www.alistapart.com:

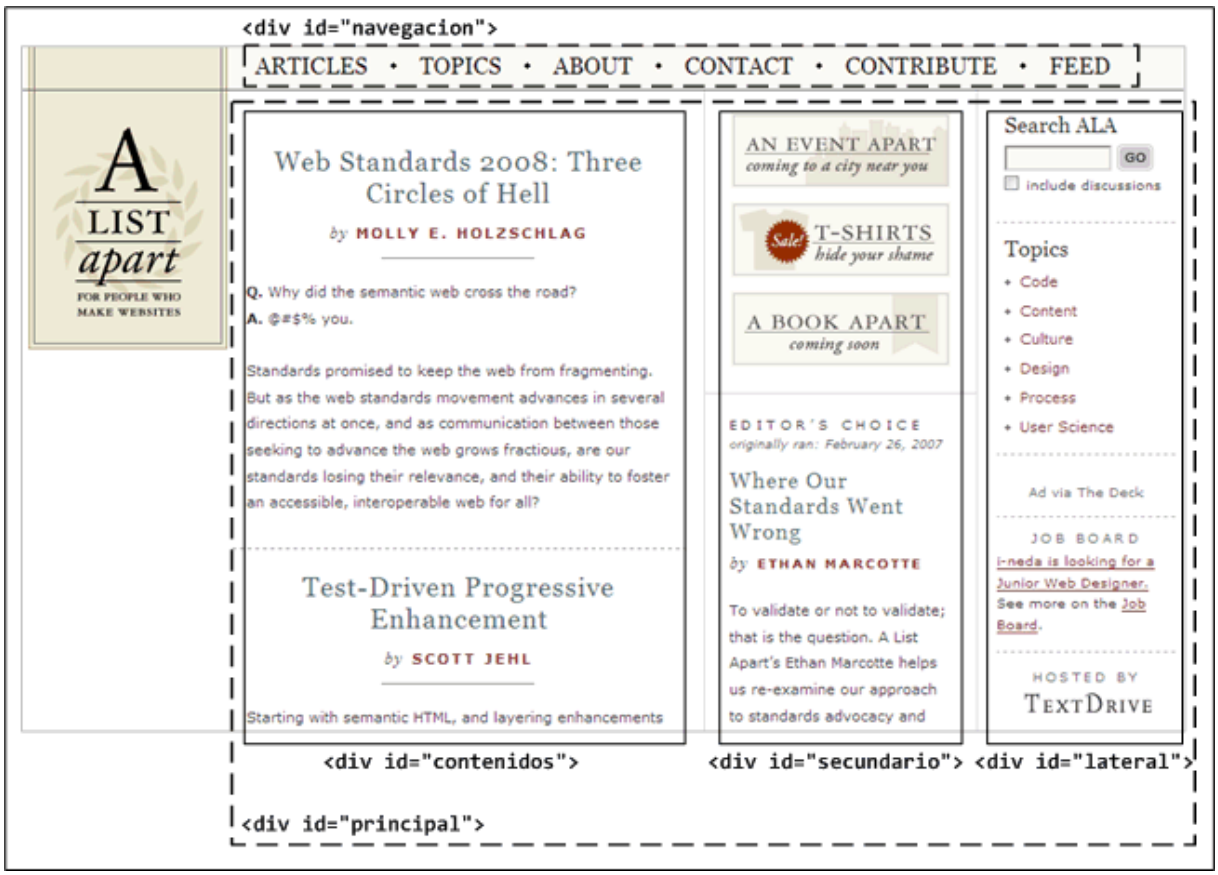


Figura 9.1. Ejemplo de página compleja estructurada con etiquetas div

Para agrupar los elementos que forman cada zona o división de la página se utiliza la etiqueta `<div>`:

	Divisiones
Atributos comunes	básicos, i18n y eventos
Atributos específicos	-
Tipo de elemento	Bloque
Descripción	Agrupa elementos de bloque

El nombre de la etiqueta `div` tiene su origen en la palabra *división*, ya que esta etiqueta define zonas o divisiones dentro de una página HTML. En cualquier caso, casi todos los diseñadores web utilizan la palabra *"capa"* para referirse a una "división". Aunque se trata de un error grave (las capas se crean mediante una propiedad de CSS llamada `z-index`) es preferible seguir llamando *"capas"* a las zonas definidas con la etiqueta `<div>` para poder entenderse con el resto de diseñadores.

Las páginas web complejas que están bien diseñadas utilizan decenas de etiquetas `<div>`. Con mucha diferencia, los atributos más utilizados con esta etiqueta son `id` (para identificar la capa de forma única) y `class` (para aplicar a la capa estilos CSS).

No se va a profundizar en el proceso de diseñar una página web mediante `<div>`, ya que no es posible diseñar una página web compleja utilizando elementos `<div>` sin utilizar hojas de estilos CSS.

Por último, si observas el código HTML de algunas páginas web complejas, verás que la mayoría utilizan los mismos nombres para identificar sus divisiones. Los nombres más comunes, y sus equivalentes en inglés, se muestran a continuación:

- `contenedor` (`wrapper`) suele encerrar la mayor parte de los contenidos de la página y se emplea para definir las características básicas de la página: su anchura, sus bordes, imágenes laterales, si se centra o no respecto de la ventana del navegador, etc.
- `cabecera` (`header`) que incluye todos los elementos invariantes de la parte superior de la página (logotipo, imagen o banner, cuadro de búsqueda superior, etc.)
- `contenido` (`content`) engloba el contenido principal del sitio (la zona de noticias, la zona de artículos, la zona de productos, etc. dependiendo del tipo de sitio web)
- `menu` (`menu`) se emplea para agrupar todos los elementos del menú lateral de navegación de la página
- `pie` (`footer`) que incluye todos los elementos invariantes de la parte inferior de la página (aviso de copyright, política de privacidad, términos de uso, etc.)
- `lateral` (`sidebar`) se emplea para agrupar los elementos de las columnas laterales y secundarias de la página.

De esta forma, el esqueleto de una página HTML compleja suele ser similar al siguiente:

```
...
<div id="contenedor">
```

```
  <div id="cabecera">
    ...
  </div>

  <div id="contenido">
    <div id="menu">
      ..
    </div>
    ...
  </div>

  <div id="pie">
    ...
  </div>
</div>
...
```

El equivalente para las páginas en inglés sería el siguiente:

```
...
<div id="wrapper">
  <div id="header">
    ...
  </div>

  <div id="content">
    <div id="menu">
      ..
    </div>
    ...
  </div>

  <div id="footer">
    ...
  </div>
</div>
```

Capítulo 10. Metainformación

Las páginas y documentos HTML incluyen más información de la que los usuarios ven en sus pantallas. Estos datos adicionales siempre están relacionados con la propia página, por lo que se denominan *metainformación* o *metadatos*. La metainformación siempre se incluye en la sección de la cabecera, es decir, dentro de la etiqueta `<head>`.

Aunque la metainformación más conocida y utilizada es el título de la propia página, se puede incluir mucha otra información útil para los navegadores y para los buscadores. En las próximas secciones se explica cómo incluir la metainformación y se introduce un concepto relacionado llamado `DOCTYPE`.

10.1. Estructura de la cabecera

Como ya se explicó anteriormente, las páginas XHTML se dividen en dos partes denominadas cabecera y cuerpo. La sección de la cabecera está formada por todas las etiquetas encerradas por la etiqueta `<head>`:

<head>	Cabecera
Atributos comunes	i18n
Atributos específicos	▪ `profile = "url"` - Especifica la URL del perfil o perfiles que utilizan los metadatos ▪ `lang = "codigo_de_idioma"` - Especifica el idioma principal de los contenidos de la página
Tipo de elemento	-
Descripción	Define la cabecera del documento HTML

La cabecera típica de una página HTML completa presenta la siguiente estructura:

```
<head>
  <!-- Zona de etiquetas META -->
  <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

  <!-- Zona de título -->
  <title>El título del documento</title>

  <!-- Zona de recursos enlazados (CSS, RSS, JavaScript) -->
  <link rel="stylesheet" href="#" type="text/css" media="screen" />
  <link rel="stylesheet" href="#" type="text/css" media="print" />

  <link rel="alternate" type="application/rss+xml" title="RSS 2.0" href="#" />

  <script src="#" type="text/javascript"></script>
</head>
```

La etiqueta `<title>` establece el título de la página. Los navegadores muestran este título como título de la propia ventana del navegador. Los buscadores utilizan este título como título de sus resultados de búsqueda.

Por tanto, el valor de `<title>` no sólo es importante para los usuarios, sino que también es importante para que los usuarios encuentren las páginas a través de los buscadores. Un error común de muchos sitios web consiste en mostrar un mismo título genérico en todas sus páginas. Cada página debe mostrar un título corto, adecuado, único y que describa inequívocamente los contenidos de la página.

Las páginas XHTML deben tener definido un título y sólo uno, por lo que todas las páginas web deben incluir obligatoriamente una etiqueta `<title>`, cuya definición formal se muestra a continuación:

<title>	Título del documento
Atributos comunes	i18n
Atributos específicos	▪ `lang = "codigo_de_idioma"` - Especifica el idioma principal del título de la página
Tipo de elemento	-
Descripción	Define el título del documento HTML

Por último, la etiqueta `<head>` permite definir en el atributo `profile` la URL de un documento externo que contiene el perfil que siguen los metadatos de la cabecera. Los blogs creados con el programa WordPress incluyen por ejemplo el siguiente perfil en su cabecera:

```
<head profile="http://gmpg.org/xfn/11">
   ...
</head>
```

El documento http://gmpg.org/xfn/11 es un perfil que define atributos adicionales para establecer la relación entre sitios web.

10.2. Metadatos

Una de las partes más importantes de la metainformación de la página son los metadatos, que permiten incluir cualquier información relevante sobre la propia página.

La especificación oficial de HTML no define la lista de metadatos que se pueden incluir, por lo que las páginas tienen libertad absoluta para definir los metadatos que consideren adecuados. La etiqueta empleada para la definición de los metadatos es `<meta>`.

<meta>	Metadatos
Atributos comunes	i18n
Atributos específicos	▪ `name = "texto"` - El nombre de la propiedad que se define (no existe una lista oficial de propiedades) ▪ `content = "texto"` - El valor de la propiedad definida (no existe una lista de valores permitidos) ▪ `http-equiv = "texto"` - En ocasiones, reemplaza al atributo "name" y lo emplean los servidores para adaptar sus respuestas al documento ▪ `scheme = "texto"` - Indica el esquema que se debe emplear para interpretar el valor de la propiedad
Tipo de elemento	-
Descripción	Permite definir el valor de los metadatos que forman la metainformación del documento

Los metadatos habituales utilizan solamente los atributos `name` y `content` para definir el nombre y el valor del metadato:

```
<meta name="autor" content="Juan Pérez" />
```

No obstante, algunas etiquetas `<meta>` muy utilizadas hacen uso del atributo `http-equiv`. Este atributo se utiliza para indicar que el valor establecido por este metadato puede ser utilizado por el servidor al entregar la página al navegador del usuario. El siguiente metadato indica al servidor que el contenido de la página es código HTML y su codificación de caracteres es UTF-8:

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
```

El atributo `scheme` no suele utilizarse, aunque permite proporcionar información de contexto para que el navegador interprete correctamente el valor del metadato. En el siguiente ejemplo, el atributo `scheme` indica al navegador que el valor del metadato hace referencia al código ISBN:

```
<meta scheme="ISBN" name="identificador" content="789-1392349610">
```

Aunque no existe una lista oficial con los metadatos que se pueden definir, algunos de ellos se utilizan en tantas páginas que se han convertido prácticamente en un estándar. A continuación se muestran los metadatos más utilizados:

Definir el autor del documento:

```
<meta name="author" content="Juan Pérez" />
```

Definir el programa con el que se ha creado el documento:

```
<meta name="generator" content="WordPress 2.8.4" />
```

Definir la codificación de caracteres del documento:

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1" />
```

Definir el copyright del documento:

```
<meta name="copyright" content="librosweb.es" />
```

Definir el comportamiento de los buscadores:

```
<meta name="robots" content="index, follow" />
```

Definir las palabras clave que definen el contenido del documento:

```
<meta name="keywords" content="diseño, css, hojas de estilos, web, html" />
```

Definir una breve descripción del sitio:

```
<meta name="description" content="Artículos sobre diseño web, usabilidad y
accesibilidad" />
```

La etiqueta que define la codificación de los caracteres (`http-equiv="Content-Type"`) se emplea prácticamente en todas las páginas y las etiquetas que definen la descripción (`description`) y las palabras clave (`keywords`) también son muy utilizadas.

10.3. DOCTYPE

El estándar XHTML deriva de XML, por lo que comparte con el muchas de sus normas y sintaxis. Uno de los conceptos fundamentales de XML es la utilización del DTD o *Document Type Definition* (*"Definición del Tipo de Documento"*).

Un DTD es un documento que recoge el conjunto de normas y restricciones que deben cumplir los documentos de un determinado tipo. Si por ejemplo se define un DTD para los documentos relacionados con libros, se puede fijar como norma que cada libro tenga un título y sólo uno, que tenga uno o más autores, que la información sobre el número de páginas pueda ser opcional, etc.

El conjunto de normas, obligaciones y restricciones que se deben seguir al crear un documento de un determinado tipo, se recogen en su correspondiente DTD. El estándar XHTML define el DTD que deben seguir las páginas y documentos XHTML. En este documento se definen las etiquetas que se pueden utilizar, los atributos de cada etiqueta y el tipo de valores que puede tener cada atributo.

En realidad, la versión 1.0 del estándar de XHTML define tres DTD diferentes. Para indicar el DTD utilizado al crear una determinada página, se emplea una etiqueta especial llamada `doctype`. La etiqueta `doctype` es el único elemento que se incluye fuera de la etiqueta `<html>` de la página. De hecho, la declaración del `doctype` es lo primero que se debe incluir en una página web, antes incluso que la etiqueta `<html>`.

Como se verá más adelante, para que una página XHTML sea correcta y válida es imprescindible que incluya el correspondiente `doctype` que indica el DTD utilizado. A continuación se muestran los tres DTD que se pueden utilizar al crear páginas XHTML:

XHTML 1.0 Estricto

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

Se trata de la variante con las normas más estrictas y las restricciones más severas. Las páginas web que incluyan este `doctype`, no pueden utilizar atributos relacionados con el aspecto de los contenidos, por lo que requiere una separación total de código HTML y estilos CSS.

XHTML 1.0 Transitorio

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

Se trata de una variante menos estricta que la anterior, ya que permite el uso de algunos atributos HTML relacionados con el aspecto de los elementos.

XHTML 1.0 Frameset

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Esta última variante la utilizan las páginas que están formadas por *frames*, una práctica completamente desaconsejada y que hoy en día sólo utilizan los sitios web obsoletos.

Si no tienes claro el DTD que más te conviene, deberías utilizar el XHTML 1.0 transitorio, ya que es más fácil crear páginas web válidas. Si tienes conocimientos avanzados de XHTML, puedes utilizar XHTML 1.0 estricto.

Por otra parte, además del `DOCTYPE` apropiado, también es necesario que las páginas web indiquen el `namespace` asociado. Un `namespace` en un documento XML permite diferenciar las etiquetas y atributos que pertenecen a cada lenguaje.

Si en un mismo documento se mezclan etiquetas de dos o más lenguajes derivados de XML (XHTML y SVG por ejemplo) y que tienen el mismo nombre, no se podría determinar a qué lenguaje pertenece cada etiqueta y por tanto, no se podría interpretar esa etiqueta o ese atributo. Los `namespaces` se indican mediante una URL.

El `namespace` que utilizan todas las páginas XHTML (independientemente de la versión y del `DOCTYPE`) es `http://www.w3.org/1999/xhtml` y se indica de la siguiente manera:

```
<html xmlns="http://www.w3.org/1999/xhtml">
...
</html>
```

De esta forma, es habitual que las páginas XHTML comiencen con el siguiente código:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="es" xml:lang="es">
```

Aunque el código anterior es mucho más complicado que una simple etiqueta `<html>`, es imprescindible para que las páginas XHTML creadas sean correctas y superen satisfactoriamente el proceso de validación que se muestra en los capítulos siguientes.

Afortunadamente, si utilizas un editor avanzado como Dreamweaver para crear las páginas, todo el código anterior se incluye de forma automática. Si creas las páginas a mano, sólo tienes que copiar y pegar ese código en cada nueva página.

Capítulo 11. Otras etiquetas importantes

11.1. Comentarios

Al igual que la mayoría de lenguajes de marcado, HTML permite incluir comentarios dentro de su código para añadir información que no se debe mostrar por pantalla.

Normalmente, los diseñadores y programadores incluyen comentarios para marcar el comienzo y el final de las secciones de las páginas, para incluir avisos y notas para otros diseñadores o para incluir explicaciones sobre la forma en la que se ha creado el código HTML.

Aunque los comentarios no se muestran por pantalla y por tanto son *invisibles* para los usuarios, sí que se descargan con el código HTML de la página. Por este motivo, nunca debe incluirse información sensible o confidencial en los comentarios.

La sintaxis de los comentarios es la siguiente:

- Apertura del comentario: `<!--`
- Contenido del comentario: `(cualquier texto)`
- Cierre del comentario: `-->`

El siguiente ejemplo muestra el uso de los comentarios HTML para indicar el comienzo y final de cada sección. Recuerda que los comentarios no se muestran por pantalla y que no influyen en la forma en la que se ven las páginas:

```
<!-- Inicio del menú -->
<div id="menu">
<ul>
  <li>...</li>
  <li>...</li>
  <li>...</li>
  <li>...</li>
</ul>
<!-- Fin del menú -->

<!-- Inicio de la publicidad -->
<div id="publicidad"> ... </div>
<!-- Fin de la publicidad -->
```

Los comentarios de HTML puede ocupar tantas líneas como sea necesario. Sin embargo, los comentarios no se pueden anidar, es decir, no se puede incluir un comentario dentro de otro comentario.

11.2. JavaScript

Como ya se explicó en los capítulos anteriores, la etiqueta `<script>` se utiliza para enlazar archivos JavaScript externos y para incluir bloques de código JavaScript en las páginas. Sin embargo, algunos navegadores no disponen de soporte completo de JavaScript, otros

navegadores permiten bloquearlo parcialmente e incluso algunos usuarios bloquean completamente el uso de JavaScript porque creen que así navegan de forma más segura.

Si JavaScript está bloqueado o deshabilitado y la página web requiere su uso para un correcto funcionamiento, es habitual incluir un mensaje de aviso al usuario indicándole que debería activar JavaScript para disfrutar completamente de la página.

El siguiente ejemplo muestra una misma página web que requiere JavaScript tanto cuando se accede con JavaScript activado y como cuando se accede con JavaScript completamente desactivado.

Imagen de www.netvibes.com con JavaScript activado

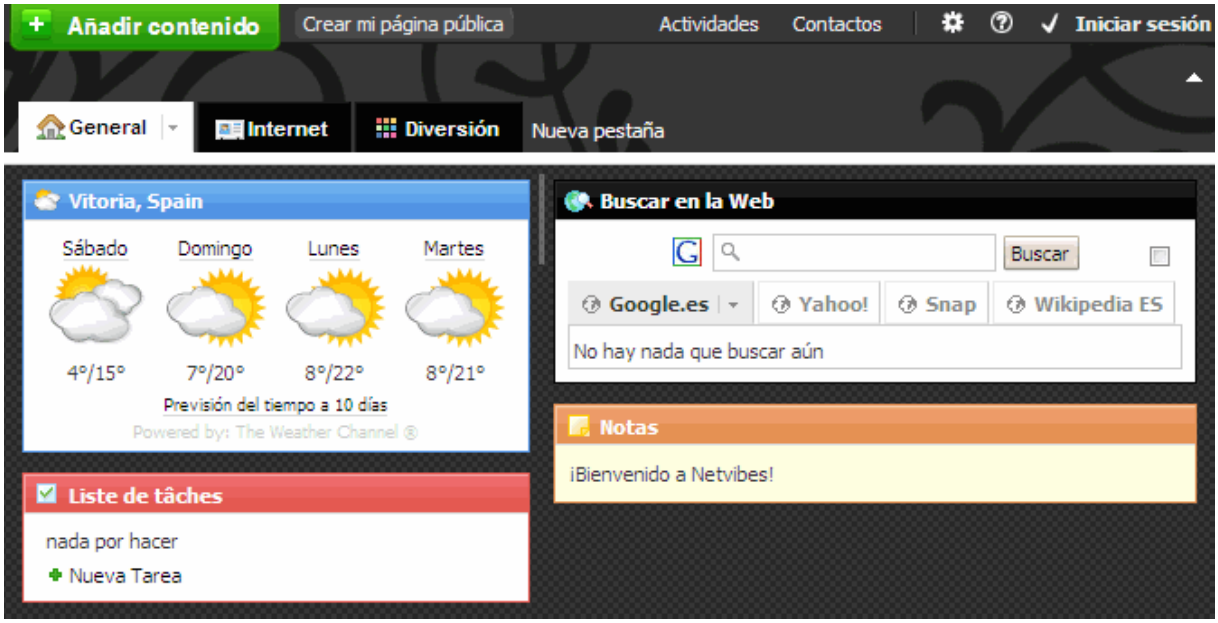


Figura 11.1. Ejemplo de página compleja con JavaScript activado

Imagen de www.netvibes.com con JavaScript deshabilitado



Figura 11.2. Ejemplo de página compleja con JavaScript desactivado

HTML define la etiqueta `<noscript>` para incluir un mensaje que los navegadores muestran cuando JavaScript se encuentra bloqueado o deshabilitado.

<noscript>	Sin soporte de scripts
Atributos comunes	básicos, i18n y eventos
Atributos específicos	-
Tipo de elemento	Bloque
Descripción	Define un mensaje alternativo que se muestra al usuario cuando su navegador no soporta la ejecución de scripts

De esta forma, incluir un mensaje de aviso que solamente sea visible en los navegadores que tienen bloqueado JavaScript es tan sencillo como incluir la etiqueta `<noscript>` dentro del `<body>`.

```
<head> ... </head>
<body>
<noscript>
  <p>Bienvenido a Mi Sitio</p>
  <p>La página que estás viendo requiere para su funcionamiento el uso de JavaScript.
Si lo has deshabilitado intencionadamente, por favor vuelve a activarlo.</p>
</noscript>
</body>
```

11.3. CSS

Algunos de los atributos más utilizados en la creación de páginas web son `id`, `class` y `style`. Los tres atributos están muy relacionados con CSS, sobre todo `class` y `style`.

El atributo `id` se emplea para asignar un identificador único a cada elemento de la página, lo que es útil tanto para aplicar estilos CSS a ese elemento como para programar aplicaciones con JavaScript.

Por otra parte, el atributo `class` se emplea para definir la clase CSS que se aplica a un elemento. La clase CSS es el nombre de un conjunto de estilos que se definen en la hoja de estilos y que se quieren aplicar a un elemento:

```
<p class="resumen">Lorem ipsum dolor sit amet, consectetuer adipiscing elit.
Maecenas at diam id enim viverra semper. Nulla id urna. Donec sodales.</p>
```

El párrafo del ejemplo anterior se muestra por pantalla con el aspecto definido por el conjunto de estilos llamado `resumen` y que se define en la hoja de estilos CSS enlazada por la página web.

El atributo `style` se emplea para definir estilos CSS directamente sobre los elementos HTML, tal y como se muestra en el siguiente ejemplo:

```
<p>Algunas palabras de esta frase se muestran de <span style="color:red">color
rojo</span></p>
```

No se debe confundir el atributo `style` con la etiqueta `<style>` que se explicó anteriormente. La etiqueta `<style>` se utiliza para incluir bloques de código CSS:

```
<head>
...
<style type="text/css">
  span {color:red;}
</style>
</head>
```

11.4. Iframes

Aunque su uso no es muy común, la etiqueta `<iframe>` puede ser muy útil en determinadas ocasiones, ya que permite insertar un documento HTML dentro de otro documento HTML. Un `iframe` puede considerarse como un *agujero* que se abre en una página web y a través del cual se muestra otra página web.

En ocasiones se utiliza para mostrar contenidos externos al sitio web como si fueran parte del mismo sitio. Otra veces se emplea para incluir una aplicación común a varios sitios web de una misma empresa.

La página principal de **Google Analytics** emplea un `<iframe>` para incluir en un pequeño recuadro la página correspondiente a la validación de usuario.



Figura 11.3. Ejemplo de página con un iframe

<iframe>	Marco (frame) en línea
Atributos comunes	básicos
Atributos específicos	▪ `src = "url"` - URL del documento HTML que se visualiza en el iframe ▪ `height = "longitud"` - Altura que ocupará el iframe en el documento ▪ `width = "longitud"` - Anchura que ocupará el iframe en el documento ▪ `name = "texto"` - Nombre que identifica al iframe ▪ `longdesc = "url"` - Dirección en la que puede encontrarse una descripción larga del contenido del iframe ▪ `scrolling = "yes | no | auto"` - Indica si el iframe debe mostrar barras de scroll (horizontal y vertical) cuando el contenido incluido no cabe en el iframe
Tipo de elemento	Bloque y en línea
Descripción	Se emplea para incluir en la página un marco que muestra otro documento HTML

El siguiente ejemplo define la altura y anchura del `iframe`, indica la URL que se debe mostrar y mediante el atributo `scrolling` se indica que el `iframe` no debe mostrar barras de *scroll* ni siquiera en el caso de que el contenido mostrado no quepa en el `iframe` definido:

```
<iframe src="/ruta/documento.html" width="250" height="250" scrolling="no" />
```

11.5. Otras etiquetas

La etiqueta `<address>` es una de las etiquetas más desconocidas de HTML, por lo que uso no está muy extendido. La etiqueta `<address>` se utiliza para proporcionar información de contacto.

<address>	Direcciones
Atributos comunes	básicos, i18n y eventos
Atributos específicos	-
Tipo de elemento	Bloque
Descripción	Define la información de contacto de un documento

El siguiente ejemplo sencillo muestra directamente el nombre, dirección y teléfono de contacto de una empresa:

```
<address>
  Nombre de la empresa
  Dirección completa
  Teléfono y Fax
</address>
```

La especificación oficial de HTML muestra un ejemplo complejo del uso de la etiqueta `<address>`:

```
<address>
  <a href="../People/Raggett/">Dave Raggett</a>,
  <a href="../People/Arnaud/">Arnaud Le Hors</a>,
  contact persons for the <a href="Activity">W3C HTML Activity</a><br/>
  $Date: 1999/12/24 23:37:50 $
</address>
```

Hasta hace unos años, la etiqueta <hr> era una de las más utilizadas, ya que permite mostrar una línea horizontal de separación. Sin embargo, hoy en día apenas se utiliza, ya que se considera un elemento puramente estético, del que no debería preocuparse HTML y para el que CSS ofrece alternativas mucho mejores.

<hr>	Línea horizontal
Atributos comunes	básicos, i18n y eventos
Atributos específicos	-
Tipo de elemento	Bloque
Descripción	Permite incluir una línea horizontal de separación

La siguiente imagen muestra el aspecto con el que los navegadores muestran por defecto las líneas horizontales creadas con <hr>:

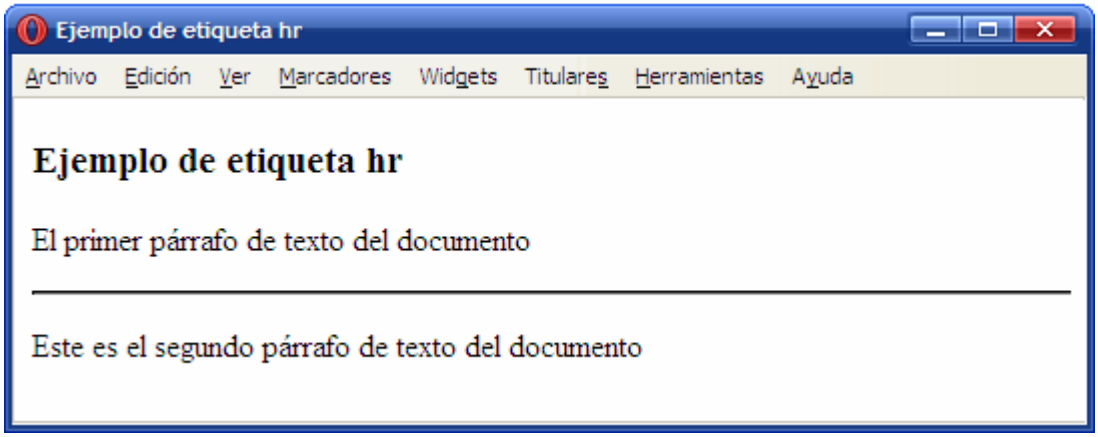


Figura 11.4. Ejemplo de uso de la etiqueta hr

El código HTML del ejemplo anterior se muestra a continuación:

```
<html>
<head><title>Ejemplo de etiqueta hr</title></head>

<body>
<h3>Ejemplo de etiqueta hr</h3>

<p>El primer párrafo de texto del documento</p>

<hr/>

<p>Este es el segundo párrafo de texto del documento</p>
```

```
</body>
</html>
```

Capítulo 12. Accesibilidad

El principal objetivo de la accesibilidad web es el de permitir a cualquier usuario, independientemente del tipo de discapacidad que sufra, el acceso a los contenidos del sitio y permitirle la navegación necesaria para realizar las acciones deseadas.

Los sitios web accesibles no solamente facilitan el acceso de sus contenidos a los usuarios discapacitados, sino que también permiten ofrecer la misma funcionalidad con dispositivos muy limitados (dispositivos sin pantalla o con pantallas minúsculas, dispositivos sin teclado ni ratón, etc.).

Las cuatro principales ventajas de diseñar un sitio web completamente accesible son las siguientes:

- Los sitios accesibles separan completamente diseño y contenido.
- Un sitio accesible puede ser accedido por multitud de dispositivos diferentes sin necesidad de reescribir el código HTML.
- Los sitios accesibles son los únicos con una audiencia potencial global, ya que permiten el acceso a todos los usuarios y a todos los dispositivos.
- Generalmente, los sitios accesibles son más fáciles de utilizar también para los usuarios sin discapacidades.

La creación de sitios accesibles puede realizarse siguiendo las recomendaciones establecidas por el W3C y que se recogen en el documento *Web Content Accessibility Guidelines* (http://www.w3.org/WAI/intro/wcag.php) (WCAG).

La versión WCAG 1.0 (http://www.w3.org/TR/WCAG10/) que se utiliza en la actualidad se publicó en 1999, mientras que la versión WCAG 2.0 (http://www.w3.org/TR/WCAG20/) se encuentra todavía en borrador y se actualizó por última vez el día 30 de abril de 2008.

Las recomendaciones del WCAG 1.0 están formadas por 65 requisitos que un sitio web debe cumplir para considerarse accesible. Los requerimientos se agrupan en prioridades.

Los requisitos de prioridad 1 son de obligado cumplimiento, los de prioridad 2 son recomendables y los de prioridad 3 son deseables. Si un sitio web cumple con todos los requisitos de prioridad 1, se considera que el sitio es conforme al nivel A de accesibilidad.

El nivel AA de accesibilidad está reservado para los sitios que cumplan todos los requisitos de prioridad 1 y prioridad 2. Finalmente, los sitios que cumplen los 65 requisitos, son conformes al nivel AAA de accesibilidad.

12.1. Requisitos del nivel A de accesibilidad

Los requisitos de accesibilidad que exige el nivel A son los siguientes:

12.1.1. Generales

1.1 Proporcionar un texto alternativo para todas las imágenes, objetos y otros elementos no textuales (mediante los atributos `alt` y `longdesc`).

2.1 Asegurar que toda la información que utilice el color como elemento informativo pueda ser entendida por las personas o dispositivos que no pueden distinguir los colores.

4.1 Marcar claramente (mediante los atributos `lang` y `xml:lang`) las variaciones del idioma del texto o de los elementos textuales (`<caption>`) respecto del idioma principal de la página.

6.1 El documento debe poder leerse completamente cuando no se utilicen hojas de estilos.

6.2 La información equivalente para los contenidos dinámicos debe adaptarse a los cambios de los contenidos dinámicos.

7.1 Ningún elemento debe parpadear en la pantalla.

14.1 El contenido del sitio se debe escribir con un lenguaje sencillo y limpio.

12.1.2. Si se utilizan mapas de imagen

1.2 Proporcionar un enlace textual por cada una de las regiones del mapa de imagen.

9.1 Utilizar mapas de imagen en el cliente, en vez de mapas de imagen de servidor.

12.1.3. Si se utilizan tablas

5.1 Utilizar cabeceras de fila y de columna.

5.2 Si la tabla tiene varios niveles de cabeceras, utilizar las agrupaciones disponibles (`<thead>`, `<tfoot>`).

12.1.4. Si se utilizan frames

12.1 Indicar un título a cada *frame* para su identificación y facilitar la navegación.

12.1.5. Si se utilizan applets y scripts

6.3 Asegurar que la página también se pueda utilizar cuando no se ejecutan los applets y los scripts. Si no es posible, proporcionar informaciones equivalente o páginas alternativas que sean accesibles.

12.1.6. Si se utilizan contenidos multimedia (audio y vídeo)

1.3 Incluir una descripción textual del contenido multimedia.

1.4 Para los contenidos basados en vídeo o animaciones, sincronizar las alternativas textuales con la presentación.

12.1.7. Si no se pueden cumplir los anteriores requisitos

11.4 Proporcionar una página alternativa con la mayor cantidad posible de contenidos y que cumpla con los requisitos anteriores.

La lista completa con los 65 requisitos de los tres niveles de accesibilidad se puede consultar en http://www.w3.org/TR/WCAG10/full-checklist.html

Capítulo 13. Validación

La validación es el proceso que asegura que un documento escrito en un determinado lenguaje (por ejemplo XHTML) cumple con las normas y restricciones de ese lenguaje. Las normas y restricciones de los documentos escritos en XML (y en sus lenguajes derivados, como XHTML) se definen en el DTD o *Document Type Definition* (*"Definición del Tipo de Documento"*).

El concepto de validación es objeto de controversia en el ámbito del diseño web. Por una parte, la validación no es obligatoria y las páginas web se pueden ver bien sin que sean válidas. Por otra parte, una página válida es más correcta que otra página que no lo sea, ya que cumple con las normas y restricciones impuestas por XHTML.

Debido a esta controversia, algunos diseñadores consideran que se da demasiada importancia a la validación de las páginas y a la creación de páginas válidas. El resto de diseñadores argumentan que si la especificación de XHTML impone una serie de normas y restricciones, lo más correcto es que las páginas web las cumplan, aunque no sea obligatorio.

En cualquier caso, el proceso de validación consiste en probar página a página si su código HTML pasa la prueba de validación. Los validadores son las herramientas que se utilizan para validar cada página. Algunos editores de páginas web incluyen sus propios validadores y el organismo W3C ha creado una herramienta gratuita para la validación de las páginas.

En las próximas secciones de este capítulo se muestran las diferentes herramientas de validación disponibles para validar las páginas web.

13.1. Validación con Dreamweaver

Si utilizas Dreamweaver para crear las páginas web, el validador se encuentra integrado en la propia herramienta. En primer lugar, accede a la configuración de la herramienta de validación desde la opción `Edición > Preferencias > Validador`:

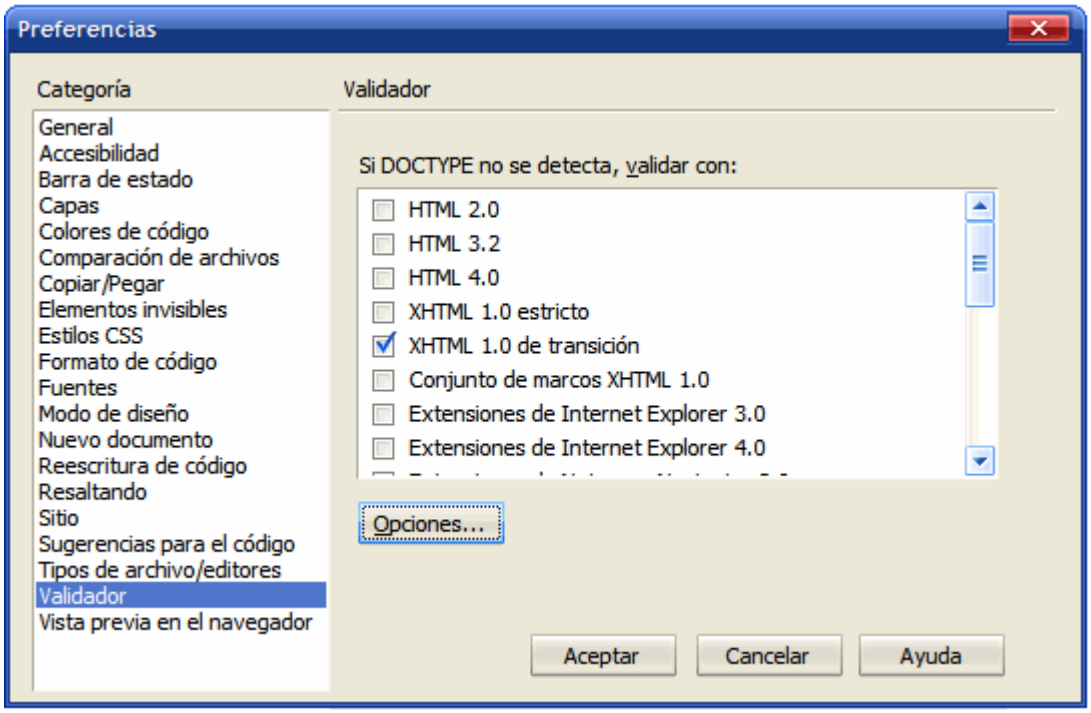


Figura 13.1. Configuración del validador de Dreamweaver

En esta ventana de configuración se puede elegir el DTD que se utiliza en caso de que la página web no indique el DTD que utiliza. Las páginas declaran el DTD que utilizan mediante el `doctype`, tal y como se explicó en capítulos anteriores. Una vez seleccionado el DTD por defecto (en la imagen anterior, se ha elegido el DTD de XHTML 1.0 de transición), se puede acceder a la herramienta de validación de Dreamweaver desde el icono que se muestra en la siguiente imagen:

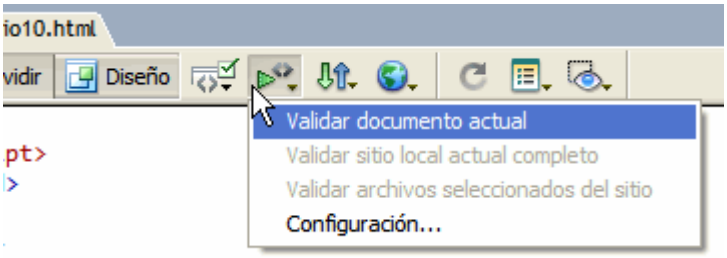


Figura 13.2. Icono que permite acceder a la herramienta de validación de Dreamweaver

Si no se han producido errores al validar la página, Dreamweaver lo indica mediante un mensaje que declara a la página como válida. Si se produce algún error, la página no es válida y Dreamweaver muestra la lista de todos los errores encontrados junto con sus posibles soluciones:

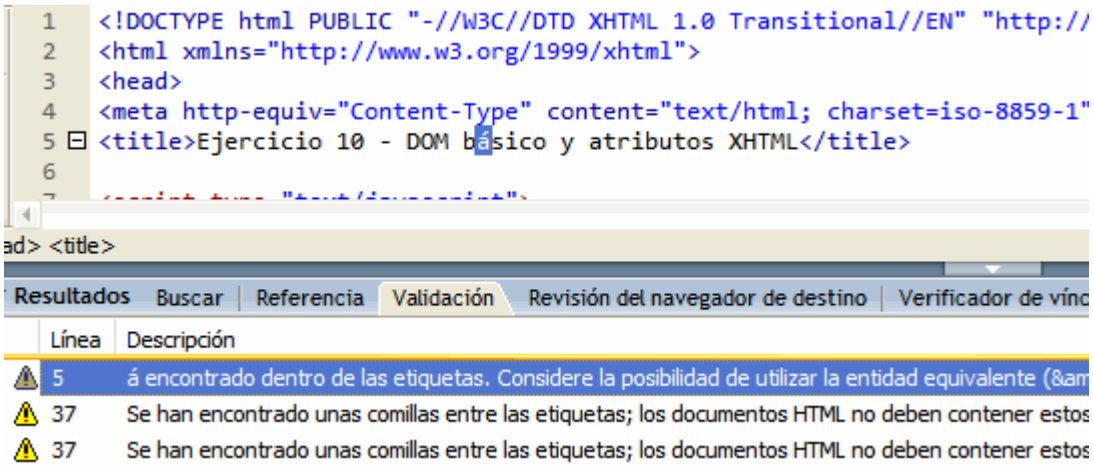


Figura 13.3. Resultado de validar una página con Dreamweaver

Después de corregir todos los errores, se puede pasar otra vez la prueba de validación para comprobar que la página cumple con todas las restricciones que impone el tipo de `doctype` que utiliza.

13.2. Validador del W3C

La validación de las páginas web no requiere el uso de editores avanzados como Dreamweaver, ya que el organismo W3C ha creado una herramienta que se puede utilizar gratuitamente a través de Internet: http://validator.w3.org/



Figura 13.4. Página principal del validador del W3C

Aunque la herramienta sólo está disponible en inglés, su uso es muy intuitivo:

- `Validate by URI`, permite escribir la URL de la página que se quiere validar. Esta opción es la más sencilla para validar las páginas que ya están publicadas en Internet.

- `Validate by File Upload`, muestra un formulario mediante el que se puede subir el archivo HTML correspondiente a la página que se quiere validar. Esta opción es la mejor para validar las páginas web que has desarrollado y que aún no has publicado en Internet.
- `Validate by Direct Input`, permite validar código HTML de forma directa. Se trata de la opción más rápida para validar *trozos* o páginas HTML completas. Esta opción es la mejor cuando estás desarrollando las páginas y quieres asegurarte que el código sea válido.

La siguiente imagen muestra el resultado de la validación de la página principal de Google realizada mediante la opción `Validate by URI`:

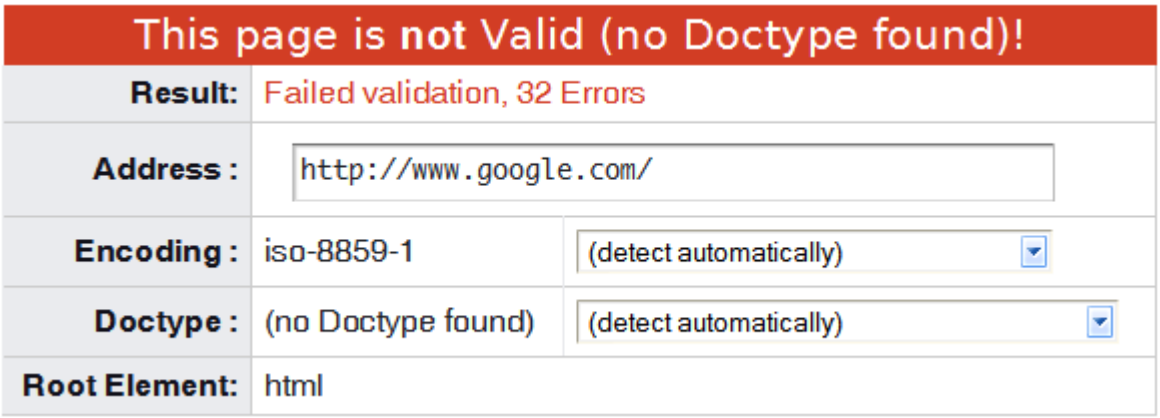


Figura 13.5. Resultado de validar una página con el validador de W3C

Si la página no pasa correctamente la prueba de validación, se muestra el listado completo de fallos junto con la ayuda necesaria para resolver cada uno de los errores.

Como se observa en la imagen anterior, incluso una página tan sencilla como la portada de Google contiene decenas de errores que impiden considerarla válida. Por lo tanto, la página principal de Google no es una página válida, aunque eso no impide que se vea bien en todos los navegadores y que los usuarios la consideren correcta.

13.3. Otros validadores

Además de los validadores disponibles en herramientas como Dreamweaver y de los validadores gratuitos disponibles en Internet, existe otro método de validación sencillo, gratuito y muy rápido. La única limitación de este validador es que necesariamente se debe utilizar el navegador Firefox.

Si ya dispones del navegador Firefox, puedes instalar el validador mediante un complemento llamado HTML Validator (https://addons.mozilla.org/es-ES/firefox/addon/249) . La instalación se realiza como cualquier otro complemento, aunque en este caso la descarga dura un poco más de lo normal porque ocupa más de 2 MB.

Tras su instalación, la primera vez que se reinicia Firefox se muestra la siguiente ventana:

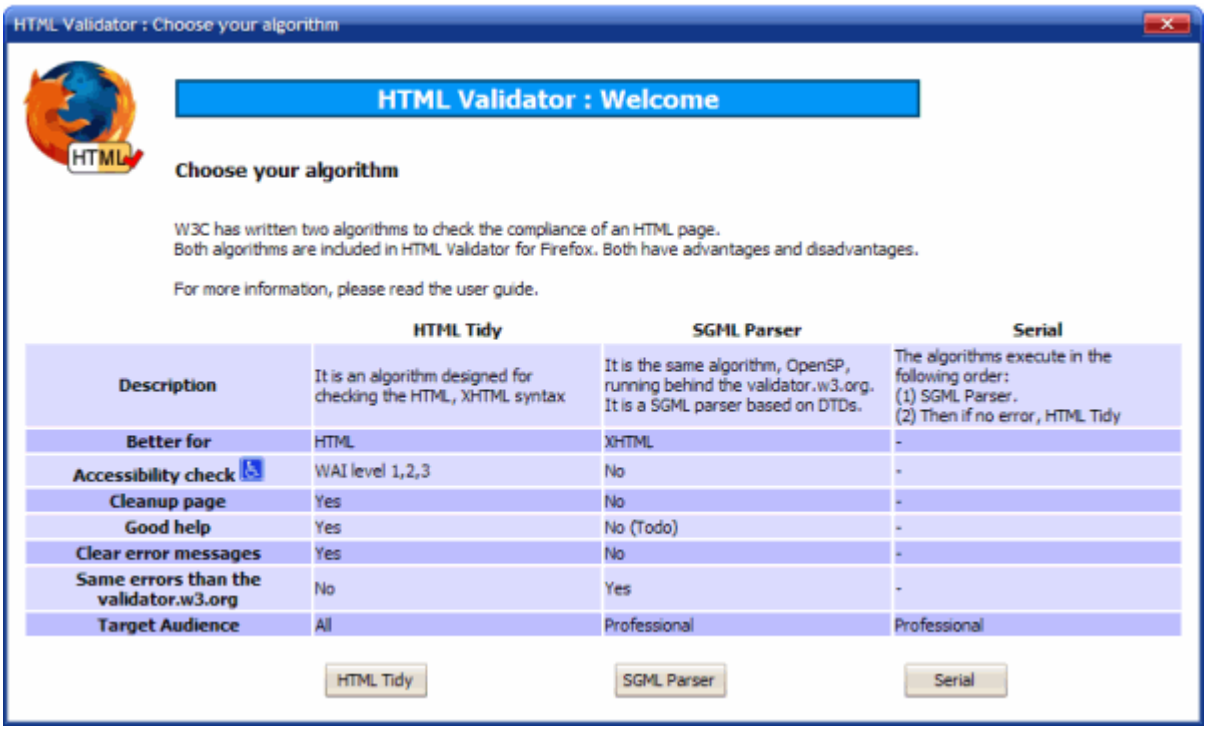


Figura 13.6. Configuración del validador Html Validator

En la ventana que se muestra se solicita al usuario que configure el tipo de validación que se va a realizar. Las opciones para elegir son: `HTML Tidy` (que ofrece ayuda para resolver los errores y es mejor para HTML), `SGML Parser` (ofrece menos ayuda, pero es el mismo que el validador del W3C) o `Serial` (que realiza las dos validaciones de forma seguida).

Si no sabes cual elegir, la opción `Serial` es una buena alternativa, ya que primero realiza la validación `SGML Parser` y a continuación, si no se han producido errores, realiza la validación `HTML Tidy`.

Una vez configurado el validador, abre cualquier página web y verás cómo en la esquina inferior derecha de Firefox se muestra un pequeño icono que indica si la página es válida o no. Cuando la página no es válida, aparece un icono correspondiente a un error. Si colocas el puntero del ratón sobre el icono, se muestra la información específica sobre los errores encontrados:



Figura 13.7. Información de error proporcionada por el validador Html Validator

Si pulsas dos veces sobre el icono, aparece una nueva ventana en la que se muestra la lista completa de errores, el lugar exacto en el que se han producido y las posibles soluciones para corregirlos.

Para ver directamente el número de errores de la página, puedes pulsar el botón derecho del ratón sobre el icono del validador y seleccionar la opción `Show` y después `Icon and Text`.

Después de activar esta opción, cada vez que cargues una página web se muestra toda la información de validación.

Aunque existen muchos otros validadores, el uso de Firefox junto con Html Validator es la única alternativa que permite validar las páginas web sin esfuerzo. Abriendo cualquier página en el navegador Firefox, es posible visualizar al instante si la página es válida o no y el número de errores que se han encontrado.

Capítulo 14. Fragmentos de código

Cuando se crean páginas web, es habitual repetir una y otra vez algunos trozos de código HTML, como por ejemplo las tablas, los formularios y la cabecera de las páginas. Para no tener que reescribir continuamente el mismo código, se utilizan los *fragmentos de código*, llamados *"snippets"* en inglés.

Crear *fragmentos de código* y trabajar con ellos es un proceso muy sencillo. A continuación se muestran los pasos necesarios para crear un fragmento correspondiente a una tabla completa de XHTML:

1. Se escribe el código XHTML completo de una tabla vacía (con sus etiquetas `<caption>`, `<thead>`, `<tbody>`, `<tfoot>`, sus atributos `summary`, `scope`, etc.)
2. Se guarda el código anterior en un archivo de texto con un nombre fácil de identificar (por ejemplo, "Tabla XHTML").
3. Cuando se necesite insertar una tabla en una página XHTML, se copia y se pega todo el código creado anteriormente y se completa con los datos necesarios.

El método descrito anteriormente no es muy eficiente si se realiza de forma manual. Afortunadamente, muchos programas utilizados para el diseño web permiten gestionar de forma más cómoda los fragmentos de código o *snippets*.

A continuación se muestra la forma de trabajar con los fragmentos de código en el conocido programa Dreamweaver:

1) Si no está visible, se debe mostrar la ventana correspondiente a los fragmentos de código mediante la opción de menú `Ventana > Fragmentos` o pulsando `Mayúsc. + F9`. Una vez abierta esta ventana, el aspecto que muestra se ve en la parte inferior izquierda de la siguiente imagen:

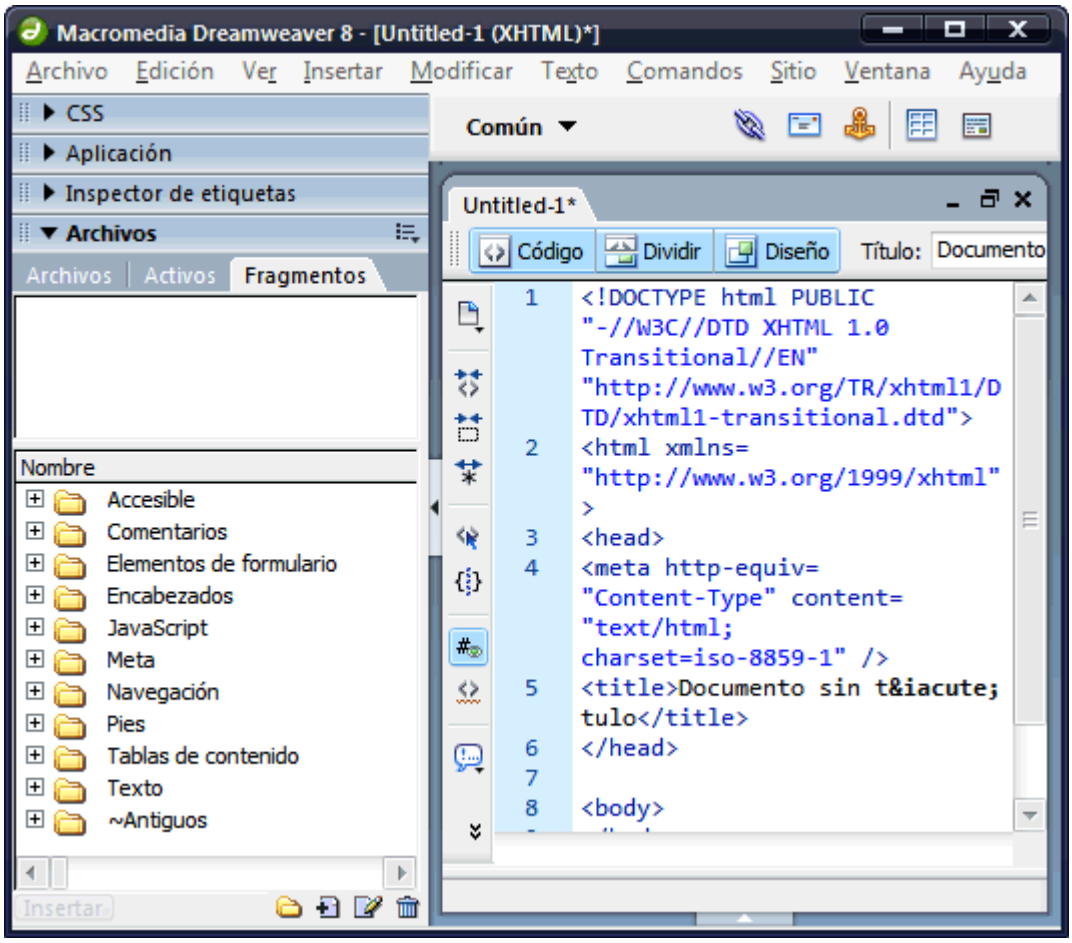


Figura 14.1. Visualizando la ventana de fragmentos de código en Dreamweaver

2) Dreamweaver ya dispone por defecto de muchos fragmentos de código útiles. Si por ejemplo se pulsa sobre la carpeta `Meta`, Dreamweaver muestra un *snippet* llamado `No crear caché`. Al pinchar dos veces sobre el nombre del fragmento, se inserta su contenido en el lugar en el que se encuentre el cursor dentro de la página, tal y como muestra la siguiente imagen:

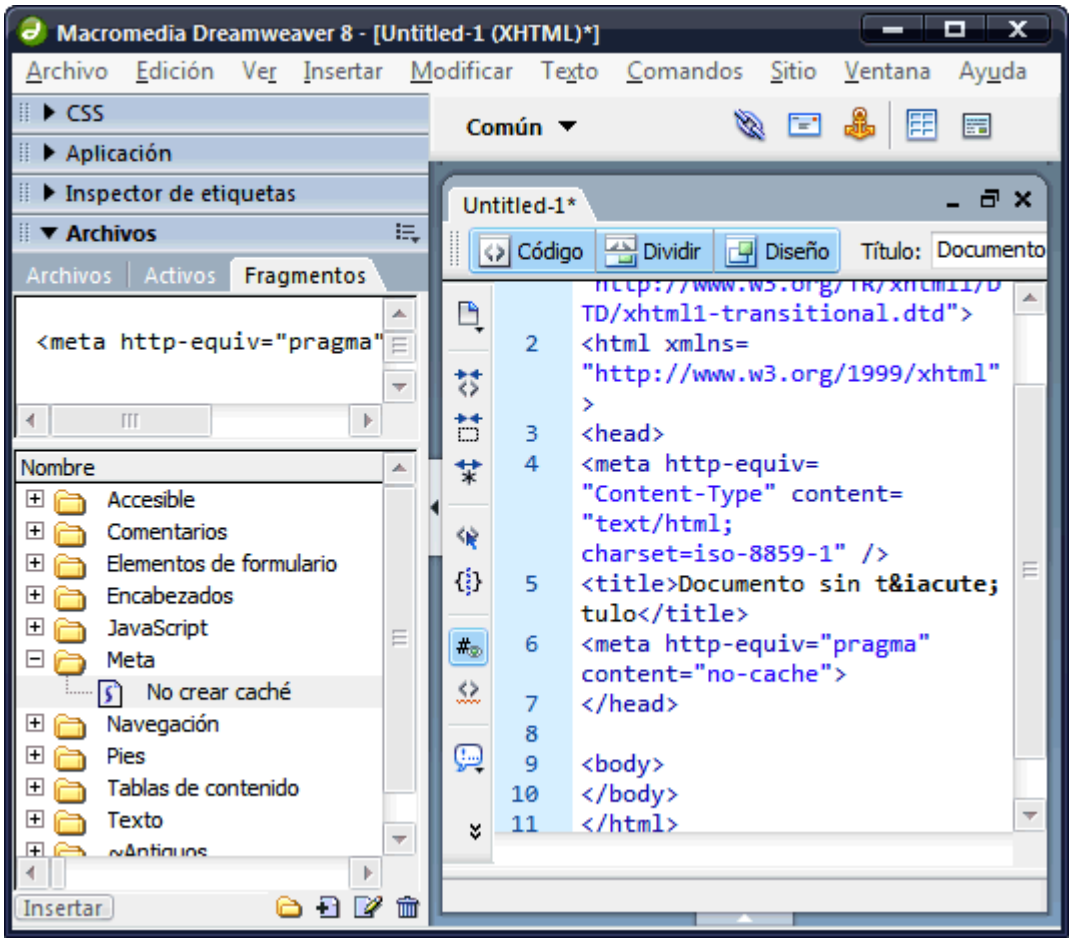


Figura 14.2. Insertando un fragmento de código de Dreamweaver en una página web

3) Para añadir un fragmento de código propio, se crea en primer lugar una carpeta en la que se guardarán todos los *snippets* propios. El nombre elegido para esta carpeta es `Propios` y la forma de crearla es pinchando sobre el pequeño icono que simboliza una carpeta:

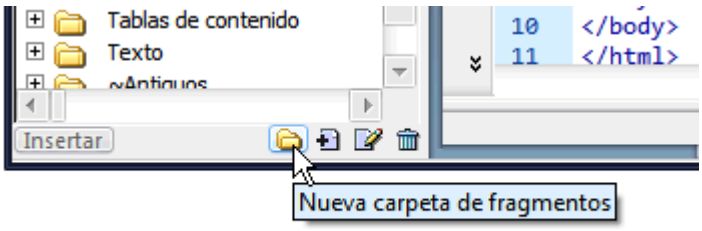


Figura 14.3. Creando una nueva carpeta para guardar los fragmentos de código propios

Una vez escrito el nombre de la nueva carpeta, el aspecto que muestra la ventana de *snippets* es el siguiente:

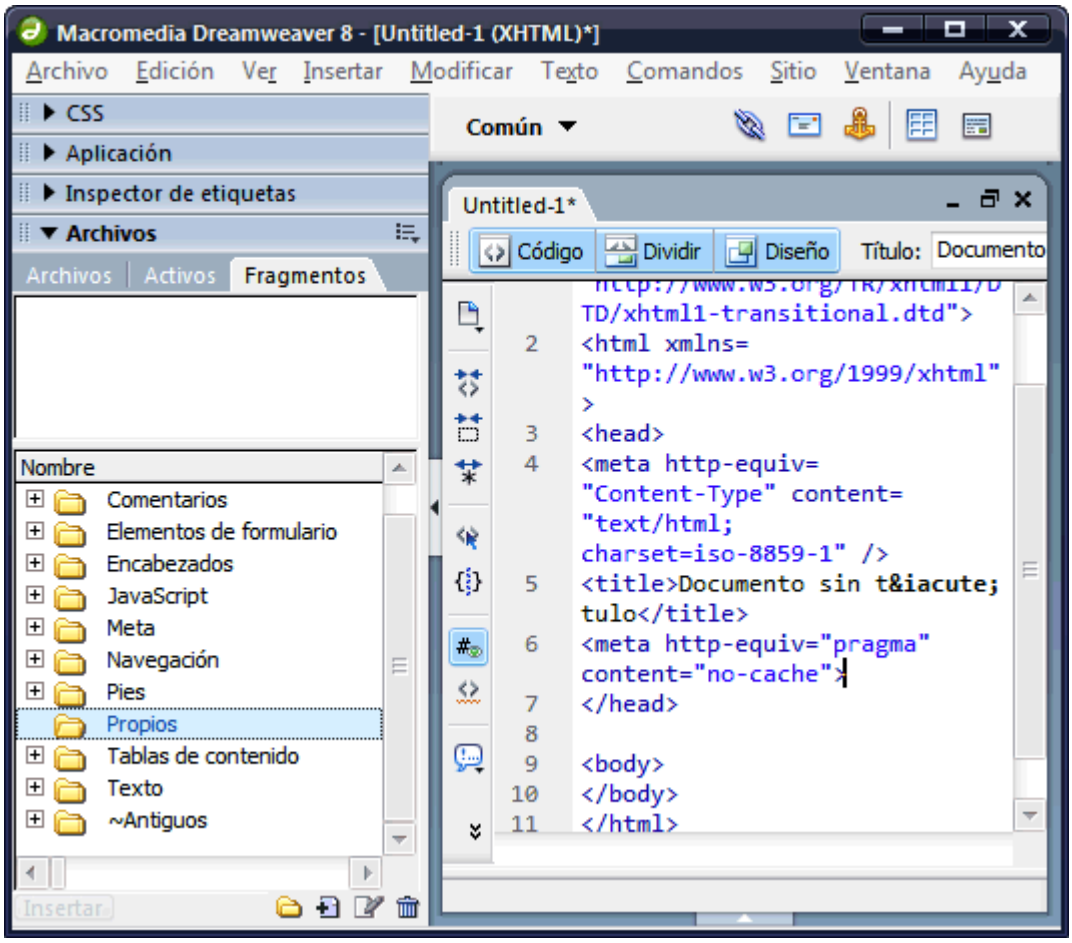


Figura 14.4. Nueva carpeta creada para guardar los fragmentos de código propios

4) Para crear un fragmento de código propio, se pulsa sobre el pequeño icono que simboliza un nuevo fragmento:

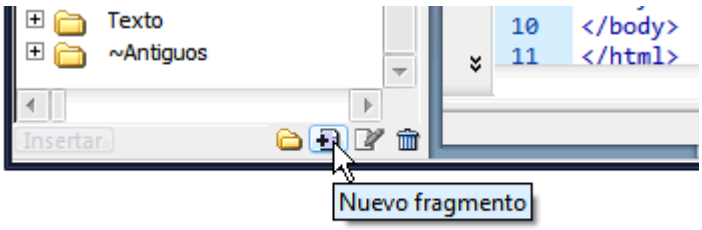


Figura 14.5. Creando un nuevo fragmento de código

Después de pulsar sobre ese icono, aparece la ventana en la que se puede escribir el nombre y el contenido del fragmento:

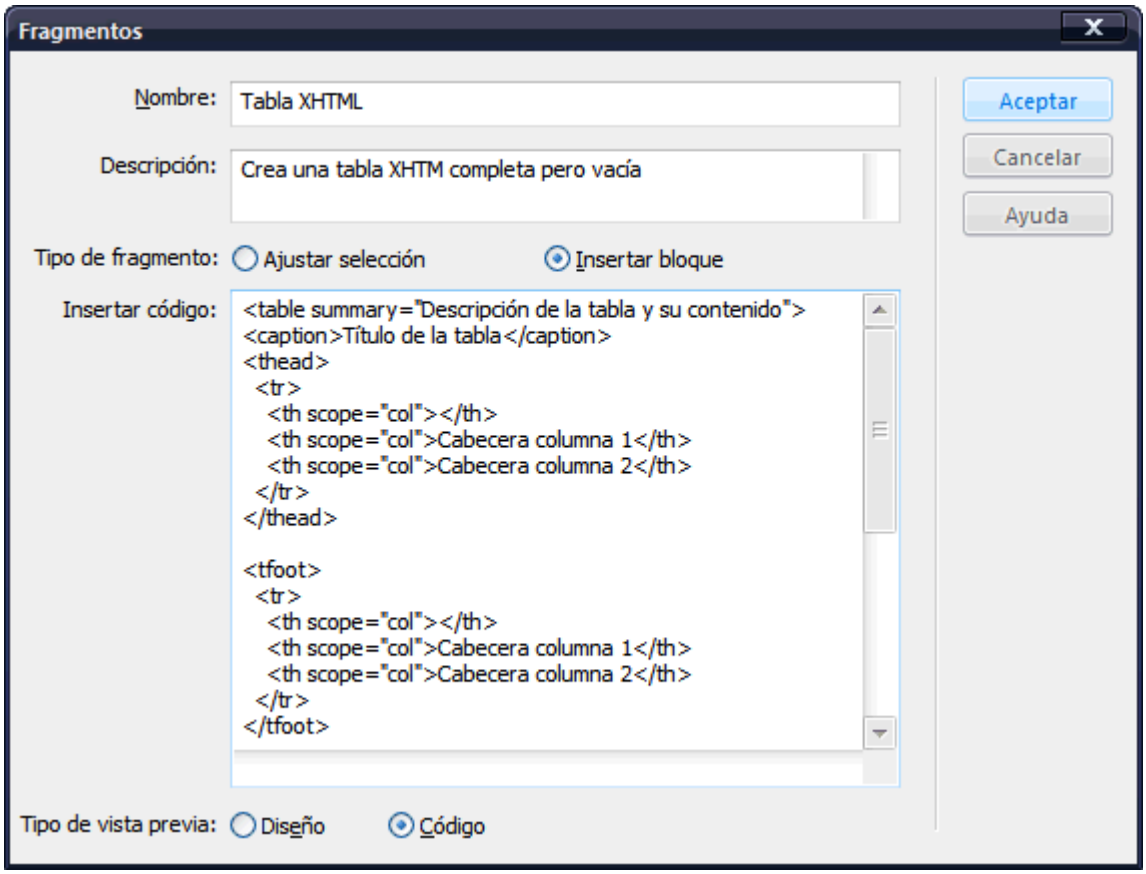


Figura 14.6. Propiedades (nombre, descripción y contenido) del fragmento de código propio

En este caso, se trata de un fragmento de código que inserta un bloque completo de código XHTML. También es posible crear fragmentos que añaden código XHTML antes y después del texto que ha sido seleccionado previamente.

Una vez creado el *snippet*, ya se puede insertar en cualquier zona de la página XHTML simplemente pulsando dos veces sobre su nombre:

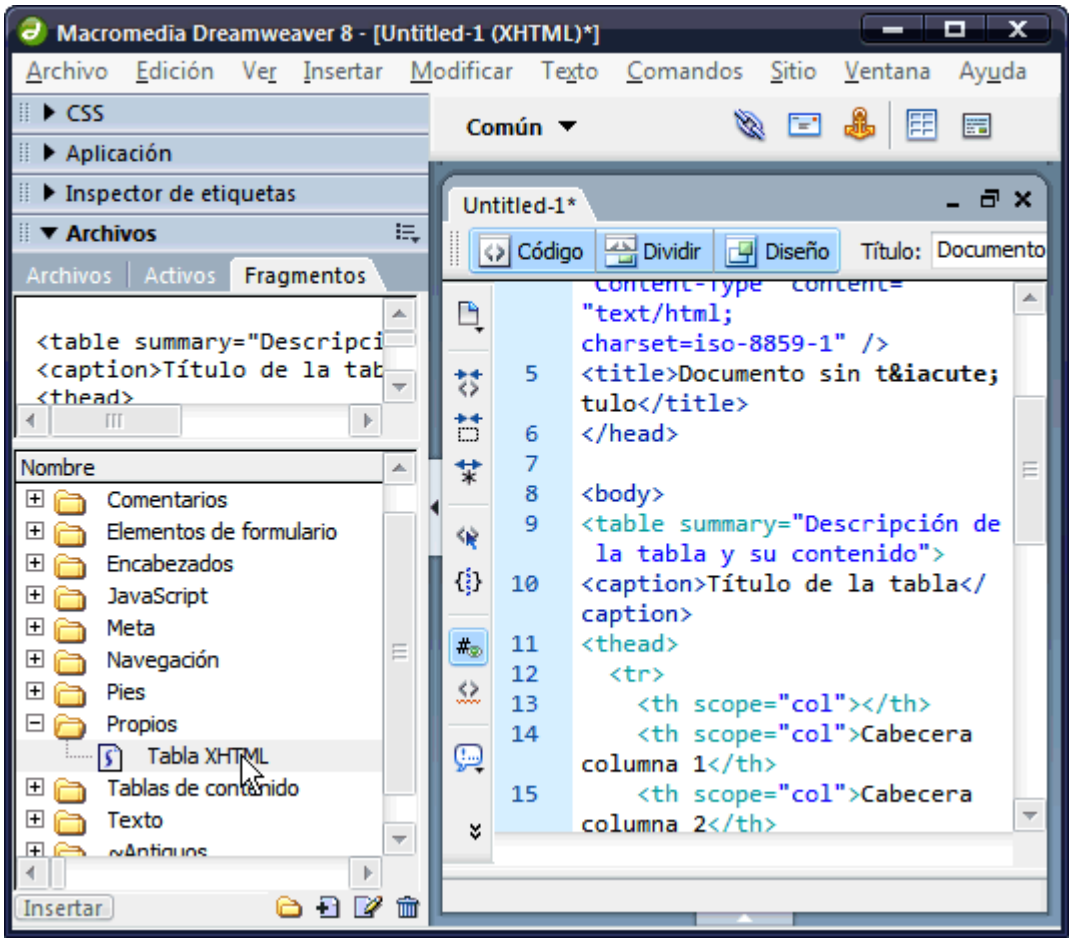


Figura 14.7. Insertando un fragmento de código propio en una página XHTML

En las siguientes secciones de este capítulo, se incluyen algunos fragmentos de código imprescindibles para crear páginas web.

14.1. Documento XHTML

La estructura básica de cualquier página o documento XHTML válido incluye al menos las siguientes etiquetas:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="es" xml:lang="es">
<head>
  <title>...</title>
  ...
</head>
<body>
  ...
</body>
</html>
```

El DOCTYPE también puede ser:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

14.2. Cabecera XHTML

Las cabeceras HTML pueden llegar a ser muy grandes, ya que incluyen toda la metainformación de la página y los enlaces a todos los recursos externos (hojas de estilos CSS, archivos JavaScript y archivos RSS).

```
<head>
<title>Título de la página</title>

<meta http-equiv="content-type" content="text/html; charset=iso-8859-1" />
<meta name="description" content="..." />
...
<meta name="author" content="..." />

<style type="text/css" media="screen,projection">
  @import '/css/archivo.css';
</style>
<link rel="stylesheet" type="text/css" href="/css/impresora.css" media="print" />
<link rel="stylesheet" type="text/css" href="/css/movil.css" media="handheld" />

<link rel="shortcut icon" href="/favicon.ico" type="image/ico" />

<link rel="alternate" type="application/rss+xml" title="..." href="/archivo_rss.xml" />

<script type="text/javascript" src="/js/archivo.js"></script>

<style type="text/css">
  /* Código CSS  */
</style>

<script type="text/javascript">
  //<![CDATA[
  // Código JavaScript
  //]]>
</script>
</head>
```

14.3. Tabla

Las tablas XHTML complejas están formadas por cabecera, pie y uno o más cuerpos. Además, es necesario incluir atributos como `summary` y `scope` para mejorar la accesibilidad.

```
<table summary="Descripción de la tabla y su contenido">
<caption>Título de la tabla</caption>
<thead>
  <tr>
    <th scope="col"></th>
    <th scope="col">Cabecera columna 1</th>
    <th scope="col">Cabecera columna 2</th>
  </tr>
</thead>

<tfoot>
  <tr>
```

```
    <th scope="col"></th>
    <th scope="col">Cabecera columna 1</th>
    <th scope="col">Cabecera columna 2</th>
  </tr>
</tfoot>

<tbody>
  <tr>
    <th scope="row">Cabecera fila 1</th>
    <td>...</td>
    <td>...</td>
  </tr>
  <tr>
    <th scope="row">Cabecera fila 2</th>
    <td>...</td>
    <td>...</td>
  </tr>
</tbody>

</table>
```

14.4. Formulario

Los formularios complejos agrupan sus campos mediante las etiquetas `<fieldset>` y `<legend>`. Además, cada campo debe incluir su título mediante la etiqueta `<label>`.

```
<form id="identificador" method="post" action="" enctype="multipart/form-data">

<fieldset>
<legend>Título del formulario</legend>

<label for="campo_texto">Campo de texto</label>
<input id="campo_texto" name="campo_texto" type="text" maxlength="255" value=""/>


<label for="campo_areatexto">Area de texto</label>
<textarea id="campo_areatexto" name="campo_areatexto" rows="10" cols="50"></textarea>

<label for="campo_desplegable">Lista desplegable</label>
<select id="campo_desplegable" name="campo_desplegable">
  <option value="0" selected="selected">- selecciona -</option>
  <option value="1">Valor 1</option>
  <option value="2">Valor 2</option>
  <option value="3">Valor 3</option>
</select>

<label for="campo_fichero">Subir un fichero</label>
<input id="campo_fichero" name="campo_fichero" type="file" value=""/>

<input type="radio" id="valor1" name="campo_radio" value="valor1" checked="checked" />
<label for="valor1">Valor 1</label>
<input type="radio" id="valor2" name="campo_radio" value="valor2" />
<label for="valor2">Valor 2</label>
```

```
<input id="campo_check" name="campo_check" type="checkbox" value="valor1"
checked="checked"/>
<label for="campo_check">Campo check 1</label>

<input id="boton_enviar" type="submit" value="Enviar formulario" />

</fieldset>
</form>
```